

Scalable Multi-Tier AI Data Ingestion Architecture

High-Throughput Autonomous Pipeline & Fault-Tolerant Processing Engine

1. Scalability & Execution Engine (500,000+ Scale)

To scale ingestion to over 500,000 startup, product, research, and signal records seamlessly without operational bottlenecks:

- **Asynchronous Web Crawling:** Built using Python's `asyncio` paired with `aiohttp` and `headful/headless` Playwright instance pools to process non-blocking asynchronous HTTP tasks and render JavaScript heavy Web applications.
- **Distributed Queue Pipeline:** Utilizes a decoupled task producer-consumer queue via `Celery` with a `Redis` back-end, allowing independent worker nodes to scale horizontally across server instances.

2. Resiliency, Fault Tolerance & LLM Routing

Failure Scenario	Architectural Mitigation Strategy
Rate Limits (HTTP 429)	Implemented exponential backoff with randomized jitter algorithms across multi-threaded workers.
Payload Limits (HTTP 413)	Integrated dynamic semantic document chunking using token-aware sliding window buffers before routing data to LLM inference pipelines.
API Provider Downtime	Multi-Tier Fallback Orchestrator (via LiteLLM): Primary route to Gemini 1.5 Flash → Automatic fallback to Groq Llama 3 8B → Secondary failover to DeepSeek API .

3. Deduplication & Data Freshness Strategy

24-Hour Deduplication Window

Every ingested record passes through a hashing pipeline that computes a deterministic **SHA-256 digest** of the normalized target URL and content payload. Hashes are stored in a distributed `Redis Key-Value Store` with a `TTL (Time-To-Live)` of 24 hours, guaranteeing zero duplicate processing within a daily ingestion cycle.

4. Storage Infrastructure & Entity Resolution

- **Relational Persistence:** Structured output schema stored in `PostgreSQL` database supporting `JSONB` columns for flexible metadata.
- **Entity Resolution Engine:** Leverages Fuzzy string matching (`Levenshtein distance`) paired with vector similarity embeddings to automatically map variant entity names (e.g., "OpenAI Inc.", "Open AI", "OpenAI") to a single canonical reference record, keeping a full audit trail log in the `entity_mapping_log` table.